

HW 01 — Floating-Point Arithmetic and Numerical Derivatives

Part A: Powers of the Golden Ratio Computed Downward

Problem Statement

Define the function `golden_powers_down(n, type)` that returns an $n \times 3$ NumPy array (in `float32` or `float64`) comparing two ways to compute the inverse powers ϕ^{-k} for $k = 0, 1, \dots, n-1$:

1. **Linear recurrence** column: $g_k = g_{k-2} - g_{k-1}$, seeded with $g_0 = 1, g_1 = 1/\phi$.
2. **Direct** column: $g_k = \phi^{-k}$ computed by Python's `**` operator.
3. **Relative difference** column: $(g_k^{\text{linear}} - g_k^{\text{direct}})/g_k^{\text{direct}}$.

Run for $n = 31$, `float32`. Plot the $\log_{10}$ of the absolute relative error vs. k .

Mathematical Background

The golden ratio $\phi = (1 + \sqrt{5})/2 \approx 1.618$ satisfies $\phi^2 = \phi + 1$. Dividing both sides by ϕ^n gives a recurrence for $g_n = \phi^{-n}$:

$$g_n = g_{n-2} - g_{n-1}$$

Verification: $g_{n-2} - g_{n-1} = \phi^{-(n-2)} - \phi^{-(n-1)} = \phi^{-n}(\phi^2 - \phi) = \phi^{-n} \cdot \phi(\phi - 1) = \phi^{-n}$, since $\phi - 1 = 1/\phi$. $\square$

The recurrence is seeded at $g_0 = 1, g_1 = 1/\phi \approx 0.618$.

Why the Recurrence Is Numerically Unstable

The general solution to $g_n = g_{n-2} - g_{n-1}$ is a linear combination of **two** basis solutions. The characteristic equation $r^2 + r - 1 = 0$ has roots $r_1 = \phi^{-1}$ and $r_2 = -\phi$, so:

$$g_n = A \cdot \phi^{-n} + B \cdot (-\phi)^n$$

In exact arithmetic with $g_0 = 1, g_1 = 1/\phi$, the coefficient $B = 0$, giving the correct decaying sequence $\phi^{-n} \rightarrow 0$.

In **floating-point**, the seeds g_0 and g_1 are slightly rounded, introducing a tiny $B \neq 0$. The spurious component $B(-\phi)^n$ grows like $\phi^n \rightarrow \infty$ while the true answer $\phi^{-n} \rightarrow 0$. The relative error therefore grows as:

$$\frac{|B(-\phi)^n|}{\phi^{-n}} = |B| \cdot \phi^{2n}$$

Each additional step multiplies the relative error by $\phi^2 \approx 2.618$. In log scale the error grows linearly with slope $2 \log_{10}(\phi) \approx 0.42$ per step.

Algorithm and Implementation

```
def golden_powers_down(n, type):
    dt = np.dtype(type) # float32 or float64
    result = np.zeros((n, 3), dtype=dt)
    phi = dt.type((1.0 + np.sqrt(5.0)) / 2.0) # cast phi to the target precision

    result[0, 0] = result[0, 1] = dt.type(1.0) # g_0 = 1
    result[1, 0] = result[1, 1] = dt.type(1.0 / phi) # g_1 = 1/phi

    for i in range(2, n):
        linear = dt.type(result[i-2, 0] - result[i-1, 0]) # g_{n-2} - g_{n-1}
        direct = dt.type(phi ** -i) # phi^{-i}
        rel_diff = dt.type((linear - direct) / direct)
        result[i] = [linear, direct, rel_diff]
    return result
```

Key detail: the dtype cast `dt.type(...)` is applied at every arithmetic operation so that all intermediate values are stored and rounded in `float32` (not promoted to `float64` silently). This faithfully reproduces the rounding errors that accumulate in restricted-precision hardware.

How It Works and What the Plot Shows

- For small k (roughly 0-7), the recurrence matches the direct formula to `float32` machine precision ($\sim 10^{-7}$), so errors scatter near -7 in log scale.
- From $k \approx 10$, the log error grows linearly with the predicted slope ≈ 0.42 .
- At $k \approx 20$, the relative error exceeds 100%—the recurrence result bears no resemblance to the true value.
- The `RuntimeWarning: divide by zero` in the log plot occurs because the relative error is exactly 0 for some early k (the two methods agree to full `float32` precision), making $\log_{10}(0) = -\infty$.

Key principle illustrated: a mathematically valid recurrence can be computationally useless if it amplifies rounding errors. The physical cause here is *catastrophic cancellation*: subtracting two nearly equal small numbers amplifies the rounding error to dominate the result.

Part B: Numerical Derivatives of $\sin(x)$

Problem Statement

Compare three finite-difference formulas for approximating $f'(x)$ where $f(x) = \sin(x)$:

Method	Formula	Truncation error order
2-point (forward)	$\frac{f(x+h)-f(x)}{h}$	$O(h)$
3-point (central)	$\frac{f(x+h)-f(x-h)}{2h}$	$O(h^2)$
5-point (central)	$\frac{-f(x+2h)+8f(x+h)-8f(x-h)+f(x-2h)}{12h}$	$O(h^4)$

Table 1: Compare errors at 10 random x values with $dx = 0.002$.

Table 2: Scan dx from 10^{-1} to 10^{-10} at $x = 0.7$.

Algorithm: Taylor-Series Derivation

All three formulas are derived by expanding $f(x \pm h)$ in a Taylor series around x :

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2}f''(x) \pm \frac{h^3}{6}f'''(x) + \frac{h^4}{24}f^{(4)}(x) \pm \dots$$

- **2-point:** $[f(x+h) - f(x)]/h = f'(x) + \frac{h}{2}f''(x) + O(h^2)$. The error (approximation minus true value) is $\approx +\frac{h}{2}f''(x)$, i.e., $O(h)$.
- **3-point:** Subtracting the expansion of $f(x-h)$ from that of $f(x+h)$ cancels the even-power terms: $[f(x+h) - f(x-h)]/(2h) = f'(x) + \frac{h^2}{6}f'''(x) + O(h^4)$. Error $\approx +\frac{h^2}{6}f'''(x)$, i.e., $O(h^2)$.
- **5-point:** Uses $f(x \pm h)$ and $f(x \pm 2h)$ with carefully chosen weights to cancel both the h^2 and h^3 error contributions: $[...]/(12h) = f'(x) - \frac{h^4}{30}f^{(5)}(x) + O(h^6)$. Error $\approx -\frac{h^4}{30}f^{(5)}(x)$, i.e., $O(h^4)$.

Implementation

```
def derivative2p(f, x, dx):
    return (f(x + dx) - f(x)) / dx

def derivative3p(f, x, dx):
    return (f(x + dx) - f(x - dx)) / (2 * dx)

def derivative5p(f, x, dx):
    return (-f(x + 2*dx) + 8*f(x + dx) - 8*f(x - dx) + f(x - 2*dx)) / (12 * dx)
```

How It Works: Explaining the Two Tables

Table 1 — Why 3-point and 5-point errors are constant across all x values:

For $f(x) = \sin(x)$, derivatives cycle: $f' = \cos x$, $f'' = -\sin x$, $f''' = -\cos x$, $f^{(5)} = \cos x$.

The *relative* error (error divided by $f'(x) = \cos x$) for the 3-point formula is:

$$\text{rel. err}_{3p} = +\frac{h^2}{6} \cdot \frac{f'''(x)}{f'(x)} = +\frac{h^2}{6} \cdot \frac{-\cos x}{\cos x} = -\frac{h^2}{6}$$

This is **independent of x** , which is why all 10 rows show the same $\text{err}_{3p} = -h^2/6 \approx -6.67 \times 10^{-7}$ at $h = 0.002$ (note the printed values are negative). Similarly:

$$\text{rel. err}_{5p} = -\frac{h^4}{30} \cdot \frac{f^{(5)}(x)}{f'(x)} = -\frac{h^4}{30} \cdot \frac{\cos x}{\cos x} = -\frac{h^4}{30} \approx -5.3 \times 10^{-13}$$

The 2-point relative error $\approx \frac{h}{2} \frac{f''(x)}{f'(x)} = -\frac{h}{2} \tan(x)$ **does** vary with x (and is negative for $x \in (0, 1)$, matching the table).

Table 2 — Two competing error sources:

As dx decreases, two effects compete:

1. **Truncation error** decreases (theory says $O(h^p)$).
2. **Round-off error** increases because subtracting nearly-equal floats loses significant digits.

The total error is minimized at an optimal dx where these balance. Estimating with machine epsilon $\varepsilon \approx 2.2 \times 10^{-16}$:

Method	Optimal dx estimate	Observed minimum
2-point	$\sqrt{2\varepsilon} \approx 2.6 \times 10^{-8}$	between 10^{-7} and 10^{-8}
3-point	$(3\varepsilon)^{1/3} \approx 9.5 \times 10^{-6}$	between 10^{-5} and 10^{-6}
5-point	$(45\varepsilon/4)^{1/5} \approx 1.3 \times 10^{-3}$	between 10^{-3} and 10^{-4}

Key insight: Higher-order methods are more accurate at moderate dx but hit round-off *sooner* (at larger dx), because their stencils involve more cancellation. For $dx < 10^{-8}$, all methods are dominated by floating-point noise.

Key Takeaways

- **A mathematically exact recurrence can be numerically useless.** $g_n = g_{n-2} - g_{n-1}$ is exact on paper, but in finite precision it amplifies a spurious growing solution $B(-\phi)^n$, so the relative error grows like ϕ^{2n} . Always ask whether a recurrence amplifies or damps rounding errors (check the roots of its characteristic equation).
- **Catastrophic cancellation** — subtracting two nearly-equal numbers — destroys significant digits and is the root cause of most floating-point instability.
- **Finite-difference accuracy is set by Taylor cancellation.** Symmetric (central) stencils cancel even-order error terms, so 2-point is $O(h)$, 3-point $O(h^2)$, 5-point $O(h^4)$.
- **There is an optimal step size.** Total error = truncation (shrinks with h) + round-off (grows as ε/h). The minimum sits where they balance; shrinking h past it makes things *worse*.
- **Higher order is not free:** higher-order stencils reach round-off-limited accuracy at a *larger* h , because they involve more cancellation.

HW 02 — Interpolation on a Non-Uniform Grid

Problem Statement

Given a non-uniformly spaced table of $\cos(x)$ on $[0, 10]$ (101 interior-perturbed points), implement and compare three interpolation methods evaluated at 10 random test points. Repeat the comparison after thinning the table to every 2nd point (~51 points) and every 4th point (~26 points). For each method and table density, report the maximum absolute error across all test points.

The non-uniform grid is constructed by taking integer indices $0, 1, \dots, 100$, rescaling to $[0, 10]$, and adding a random perturbation $\pm \frac{1}{3}$ of the nominal spacing to each interior point.

The Three Interpolation Methods

Method 1: Linear Interpolation — $O(h^2)$

What it is: Given the test point x_1 , find the interval $[x_i, x_{i+1}]$ containing it, then linearly connect the two endpoint values:

$$y_1 = y_i + (y_{i+1} - y_i) \frac{x_1 - x_i}{x_{i+1} - x_i}$$

Implementation:

```
i = np.searchsorted(x, x1, side='right') - 1  # find left bracket index
i = int(np.clip(i, 0, n - 2))
return float(y[i] + (y[i+1] - y[i]) * (x1 - x[i]) / (x[i+1] - x[i]))
```

`np.searchsorted` performs binary search in $O(\log n)$ time to locate the bracket.

Error: The leading-order truncation error comes from the neglected curvature: $\varepsilon \approx \frac{1}{8}h^2 f''(\xi)$, where h is the local spacing. When the spacing doubles, the error grows by a factor of ≈ 4 .

Method 2: Lagrange 4-Point Interpolation — $O(h^4)$

What it is: Fit a cubic polynomial through 4 nodes $x_{i-1}, x_i, x_{i+1}, x_{i+2}$ bracketing x_1 (2 nodes on each side). The interpolant is the unique degree-3 polynomial passing through all four:

$$y_1 = \sum_{k=0}^3 y_{n_k} L_k(x_1), \quad L_k(x_1) = \prod_{j \neq k} \frac{x_1 - x_{n_j}}{x_{n_k} - x_{n_j}}$$

L_k is the Lagrange basis polynomial: it equals 1 at x_{n_k} and 0 at all other nodes.

Implementation:

```
i = int(np.searchsorted(x, x1, side='right') - 1)
# Boundary rule: return table value near edges
if x1 <= x[1]: return float(y[1])
if x1 >= x[n-2]: return float(y[n-2])
i = int(np.clip(i, 1, n - 3)) # ensure [i-1, i, i+1, i+2] are valid
xn = x[[i-1, i, i+1, i+2]]
```

```

yn = y[[i-1, i, i+1, i+2]]

result = 0.0
for k in range(4):
    Lk = 1.0
    for j in range(4):
        if j != k:
            Lk *= (x1 - xn[j]) / (xn[k] - xn[j])
    result += yn[k] * Lk

```

Error: The truncation error is $O(h^4)$: the first neglected Taylor term involves $f^{(4)}(x)$. When spacing doubles, error grows by ≈ 16 .

Boundary handling: When $x_1 \leq x[1]$ or $x_1 \geq x[n-2]$, there aren't 2 nodes on both sides, so the code falls back to the nearest table value.

Method 3: Hermite 4-Point Interpolation — $O(h^4)$

What it is: A cubic spline on the interval $[x_i, x_{i+1}]$ that matches both the function **values** and the first **derivatives** at the two endpoints. This guarantees a C^1 (continuously differentiable) interpolant — no kinks at the breakpoints.

The cubic Hermite basis (using parameter $t = (x_1 - x_i)/h$, $h = x_{i+1} - x_i$):

$$H(x_1) = (2t^3 - 3t^2 + 1) y_i + (t^3 - 2t^2 + t) h m_i + (-2t^3 + 3t^2) y_{i+1} + (t^3 - t^2) h m_{i+1}$$

where m_i and m_{i+1} are the estimated derivatives at the two endpoints.

Derivative estimation: Since the grid is non-uniform, simple central differences give only $O(h)$ accuracy. Instead, the 3-point non-uniform derivative formula (derived from the Lagrange polynomial through x_a, x_m, x_b) gives $O(h^2)$:

$$f'(x_m) = f_a \frac{x_m - x_b}{(x_a - x_m)(x_a - x_b)} + f_m \frac{2x_m - x_a - x_b}{(x_m - x_a)(x_m - x_b)} + f_b \frac{x_m - x_a}{(x_b - x_a)(x_b - x_m)}$$

For m_i , the three nodes used are x_{i-1}, x_i, x_{i+1} ; for m_{i+1} , they are x_i, x_{i+1}, x_{i+2} .

Implementation:

```

def deriv3p(xa, xm, xb, fa, fm, fb):
    return (fa*(xm-xb)/((xa-xm)*(xa-xb))
            + fm*(2*xm-xa-xb)/((xm-xa)*(xm-xb))
            + fb*(xm-xa)/((xb-xa)*(xb-xm)))

mi = deriv3p(xn[0], xn[1], xn[2], yn[0], yn[1], yn[2])
mi1 = deriv3p(xn[1], xn[2], xn[3], yn[1], yn[2], yn[3])
h = xn[2] - xn[1]
t = (x1 - xn[1]) / h
return ((2*t**3 - 3*t**2 + 1)*yn[1] + (t**3 - 2*t**2 + t)*h*mi
        + (-2*t**3 + 3*t**2)*yn[2] + (t**3 - t**2)*h*mi1)

```

Error: Also $O(h^4)$, same order as Lagrange 4-point. In practice the error constant is different because the cubic Hermite and the Lagrange cubic use different approaches to fit the data.

Results and Analysis

Observed Error Summary

Table	Linear max err	Lagrange4 max err	Hermite4 max err
101 pts	1.6×10^{-3}	3.6×10^{-6}	1.1×10^{-5}
51 pts	6.4×10^{-3}	5.0×10^{-5}	1.1×10^{-4}
26 pts	1.8×10^{-2}	5.4×10^{-4}	7.7×10^{-4}

Convergence rates: When spacing doubles (halving the number of points from 101 to ~51): - Linear: error grows by $\sim 4\times$ — consistent with $O(h^2)$. - Lagrange4 and Hermite4: error grows by $\sim 10\text{--}14\times$ — consistent with $O(h^4)$ (the ideal factor is 16; the ratio is approximate because the grid is randomly perturbed and thinning does not exactly double every local spacing).

Why Hermite can be slightly worse than Lagrange here: Hermite requires estimating derivatives numerically (via the 3-point formula), which introduces additional approximation error. The net truncation constant can be larger than Lagrange’s at the same order, especially on coarse grids where the derivative estimate is less accurate.

Log-log slope on the plot equals the order: A log-log plot of max error vs. number of table points should show slopes of -2 (linear) and -4 (both 4-point methods), confirming the theoretical orders.

Key Takeaways

- **Interpolation order = number of points used.** Linear (2 points) is $O(h^2)$; cubic methods (4 points) are $O(h^4)$. The error order is the slope on a log-log error-vs-density plot.
- **Halving the table density** multiplies the error by 2^{order} : $\times 4$ for linear, $\times 16$ for the cubic methods. This is the practical signature of the convergence order.
- **Lagrange interpolation** fits the unique polynomial through the nodes via basis polynomials L_k (each is 1 at its node, 0 at the others). Simple and accurate, but only C^0 across intervals.
- **Hermite interpolation** matches values *and* derivatives at the endpoints, producing a smooth C^1 curve. The derivatives must themselves be estimated — here via a 3-point formula.
- **Non-uniform grids need non-uniform formulas.** A naive central difference assumes equal spacing and drops to $O(h)$; the proper 3-point Lagrange-derivative formula restores $O(h^2)$.
- **Bracketing is $O(\log n)$** via binary search (searchsorted); higher-order methods need special handling near the boundaries where there aren’t enough neighbors.

HW 03 — Numerical Integration and Root-Finding: Quantum Levels of O₂

Overview

This assignment builds two general-purpose numerical tools — an adaptive integrator and a bracketed root-finder — and applies them to compute the quantized vibrational energy levels of the O₂ molecule using the **Bohr-Sommerfeld-Wilson (BSW)** quantization rule in both the harmonic approximation and the full Lennard-Jones potential.

Task 1: Numerical Integration — f_integral

Problem Statement

Implement `f_integral(f, a, b, eps, global_type, integral_type)` that computes $\int_a^b f(x) dx$ using two quadrature rules (`simpson`, `gauss`) and two global strategies (`fix_segments`, `recursive`). Validate on $\int_0^\pi \sin x dx = 2$.

The Two Quadrature Rules

Both rules approximate $\int_a^b f(x) dx$ on a single small interval.

Simpson's rule: Approximates f by a parabola through the left endpoint, midpoint, and right endpoint:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \left[\frac{f(a)}{3} + \frac{4f(m)}{3} + \frac{f(b)}{3} \right], \quad m = \frac{a+b}{2}$$

Uses 3 function evaluations. Truncation error $O(h^5)$ per interval, $O(h^4)$ globally.

3-point Gauss-Legendre rule: Places evaluation points at $\pm\sqrt{3/5}$ and 0 on $[-1, 1]$, then maps to $[a, b]$:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \left[\frac{5}{9}f(x_1) + \frac{8}{9}f(x_2) + \frac{5}{9}f(x_3) \right]$$

where $x_1 = m - \sqrt{0.6} \cdot h$, $x_2 = m$, $x_3 = m + \sqrt{0.6} \cdot h$. Also 3 evaluations, but the node placement is optimal: Gauss-Legendre with n points integrates polynomials of degree $\leq 2n-1$ exactly. For $n=3$ this is degree 5, same order as Simpson — but with a smaller constant.

Implementation of a single segment:

```
def _integrate_segment(f, a, b, integral_type):
    mid = 0.5 * (a + b); half = 0.5 * (b - a)
    if integral_type == 'simpson':
        return half * (f(a)/3 + 4*f(mid)/3 + f(b)/3)
    elif integral_type == 'gauss':
        sq = np.sqrt(0.6)
        return half * (5/9*f(mid - sq*half) + 8/9*f(mid) + 5/9*f(mid + sq*half))
```


The Two Global Strategies

fix_segments: Divides $[a, b]$ into $n = \lfloor 1/\varepsilon \rfloor$ equal subintervals and applies the quadrature rule to each. Work is fixed regardless of where the integrand varies.

```
n = max(1, int(1.0 / eps))
xs = np.linspace(a, b, n + 1)
total = sum(_integrate_segment(f, xs[i], xs[i+1], integral_type) for i in range(n))
```

recursive (adaptive): Starts with the whole interval. On each sub-interval, compares the one-piece estimate I_{full} against the two-piece estimate I_{half} . If $|I_{\text{full}} - I_{\text{half}}|$ exceeds the local tolerance (proportional to the sub-interval length), it subdivides and recurses. This concentrates work where the integrand is varying rapidly.

```
def _recurse(lo, hi, depth):
    I_full = _integrate_segment(f, lo, hi, integral_type)
    mid = 0.5*(lo + hi)
    I_half = _integrate_segment(f, lo, mid, integral_type) + _integrate_segment(f, mid, hi, integral_type)
    tol = eps * (hi - lo) / full_length
    if depth >= 50 or (depth >= 2 and abs(I_full - I_half) <= tol):
        return I_half
    return _recurse(lo, mid, depth+1) + _recurse(mid, hi, depth+1)
```

Results: $\sin(x)$ from 0 to π

Strategy	Rule	Error	Function calls
fix_segments	simpson	$\sim 10^{-16}$	30 000
fix_segments	gauss	$\sim 10^{-16}$	30 000
recursive	simpson	$\sim 10^{-6}$	135
recursive	gauss	$\sim 10^{-9}$	63

recursive+gauss achieves the best accuracy with the fewest calls because Gauss nodes are optimally placed and the adaptive strategy avoids over-sampling the smooth $\sin x$ integrand.

Task 2: Root-Finding — x_{root}

Problem Statement

Implement $x_{\text{root}}(f, a, b, \text{epsx}, \text{epsf}, \text{type})$ that finds a root of f in $[a, b]$ where $f(a)$ and $f(b)$ have opposite signs. Implement bisection and secant methods, stopping when the bracket narrows to epsx or $|f(x)| < \varepsilon_f$.

Bisection Method

Algorithm: Repeatedly halve the bracket. If $f(\text{mid})$ and $f(a)$ have the same sign, the root is in $[\text{mid}, b]$; otherwise it's in $[a, \text{mid}]$.

```
while True:
    mid = 0.5 * (lo + hi)
    fmid = f(mid)
    if abs(fmid) < epsf or (hi - lo) < epsx:
        return mid
```

```

if flo * fmid < 0:
    hi = mid
else:
    lo, flo = mid, fmid

```

Convergence: Linear — each step halves the interval width. Guarantees convergence but is slow: needs $\sim \log_2((b-a)/\varepsilon_x)$ steps.

Secant Method (with bisection fallback)

Algorithm: Fits a straight line through the last two points (x_0, f_0) and (x_1, f_1) and extrapolates to find the next estimate:

$$x_2 = x_1 - f_1 \cdot \frac{x_1 - x_0}{f_1 - f_0}$$

If the result lies outside the current bracket $[lo, hi]$, it falls back to bisection. The bracket is updated so it always contains the root.

```

x2 = x1 - f1*(x1 - x0)/(f1 - f0)
if x2 < lo or x2 > hi:           # unsafe: use bisection
    x2 = 0.5*(lo + hi)
f2 = f(x2)
if flo * f2 < 0: hi, fhi = x2, f2
else:           lo, flo = x2, f2
x0, f0 = x1, f1;  x1, f1 = x2, f2

```

Convergence: Superlinear (order ≈ 1.618 , the golden ratio) when well-behaved. The bisection fallback prevents the secant from escaping the bracket.

Result on $\cos(x) = 0$ **in** $[0.1, \pi/2 + 0.1]$: Bisection needs 33 calls; secant needs only 7.

Task 3: Harmonic Oscillator Verification

Problem Statement

For $v(x) = x^2$ and $\gamma = 1$, the BSW quantization condition is:

$$\gamma \int_{x_1}^{x_2} \sqrt{\varepsilon - v(x)} dx = \left(n + \frac{1}{2}\right) \pi$$

where $x_{1,2} = \mp\sqrt{\varepsilon}$ are the classical turning points. This must be solved for ε_n for $n = 0, 1, 2, 3, 4$.

For the harmonic oscillator, the exact answer is $\varepsilon_n = 2n + 1$.

How It Works

Define $s(\varepsilon) = \gamma \int_{x_1}^{x_2} \sqrt{\varepsilon - v(x)} dx$ and root-find $f(\varepsilon) = s(\varepsilon) - (n + 1/2)\pi = 0$.

The integrand $\sqrt{\varepsilon - x^2}$ has **integrable square-root singularities** at the turning points (where it equals zero). To avoid roundoff at exactly the turning point, the integration limits are shifted inward by $\delta = 10^{-9}(x_2 - x_1)$:

```
ff = lambda x: np.sqrt(max(0., energy - v_sq(x)))
return gamma * f_integral(ff, x_1 + delta, x_2 - delta, 1e-6, 'recursive', 'gauss')
```

Result: Relative errors $< 10^{-9}$, confirming the code reproduces the known harmonic-oscillator energies $\varepsilon_n = 1, 3, 5, 7, 9$.

Task 4: Harmonic Approximation to the Lennard-Jones Potential

Near the minimum $x_{\min} = 2^{1/6}$, the LJ potential is approximated by a parabola:

$$v_{\text{hr}}(x) = -1 + \frac{1}{2}k(x - x_{\min})^2, \quad k = v''(x_{\min}) = 4 \left(\frac{156}{x_{\min}^{14}} - \frac{42}{x_{\min}^8} \right)$$

The turning points are $x_{1,2} = x_{\min} \mp \sqrt{2(\varepsilon + 1)/k}$. With $\gamma = 150$, this yields 14 bound states with nearly **constant spacing** ≈ 0.0713 (as expected for a harmonic well).

Task 5: Full Lennard-Jones Potential

$$v(x) = 4 \left(\frac{1}{x^{12}} - \frac{1}{x^6} \right)$$

The exact turning points for energy ε are solved analytically from $v(x) = \varepsilon$:

$$x_{1,2} = \left(\frac{2}{1 \pm \sqrt{1 + \varepsilon}} \right)^{1/6}$$

With $\gamma = 150$, the code finds 39 bound states. Key observations: - Level spacing **decreases** at high n (real well becomes softer near $\varepsilon = 0$). - The harmonic approximation matches well for low n (small-amplitude oscillations near the minimum) but diverges at high n where the asymmetry of the true potential matters. - More function calls are needed for high- n states because the outer turning point moves far out into the flat part of the potential.

Task 7: Effect of Accuracy Parameter on ε_{18}

Repeats the LJ calculation for $n = 18$ with ep ranging from 10^{-3} to 10^{-12} . The integration tolerance is set to $10^4 \cdot \text{ep}$ and the root tolerance to ep .

Result: The computed ε_{18} converges to ≈ -0.1665468 as ep decreases. Below $\text{ep} \approx 10^{-9}$, the answer is stable but each tighter tolerance costs significantly more function calls. This illustrates the trade-off: accuracy is not free, and there is a point of diminishing returns where further refinement is not worth the cost.

Task 8: Comparison with SciPy

Repeats Task 5 using `scipy.integrate.quad` and `scipy.optimize.brentq` in place of the custom functions.

`quad` uses adaptive Gaussian quadrature with built-in error control. `brentq` is a hybrid of bisection, secant, and inverse quadratic interpolation. The differences between the custom and SciPy results are $\lesssim 10^{-7}$, confirming the custom implementation is correct. SciPy typically uses fewer function calls because its error estimators are more refined.

Key Takeaways

- **Gauss-Legendre beats Newton-Cotes per function call.** With n points, Gauss integrates polynomials up to degree $2n - 1$ exactly (degree 5 for 3 points), versus degree 3 for 3-point Simpson — same cost, smaller error.
- **Adaptive (recursive) quadrature** concentrates effort where the integrand varies, comparing a one-piece against a two-piece estimate and subdividing only when they disagree. It is far cheaper than fixed sampling for smooth or locally-varying functions.
- **Bisection is slow but bulletproof** (guaranteed linear convergence); the **secant method** is superlinear (order ≈ 1.618) but can escape the bracket — so a robust solver uses secant with a **bisection fallback**.
- **A bracketed root requires a sign change** $f(a)f(b) < 0$; the bracket must be maintained so it always contains the root.
- **Integrable singularities** (here $\sqrt{\varepsilon - v}$ at the turning points) are handled by nudging the integration limits inward by a tiny δ and letting the adaptive integrator resolve the steep endpoint behavior.
- **The BSW quantization rule** $\gamma \int_{x_1}^{x_2} \sqrt{\varepsilon - v} dx = (n + \frac{1}{2})\pi$ turns “find the energy levels” into a root-find-inside-an-integral — a recurring pattern in computational physics.
- **Tolerance has diminishing returns:** past a point, tightening it only multiplies the cost without changing the answer. Always validate a custom routine against a trusted library (`quad`, `brentq`).

HW 04 — White Dwarf Structure via Multi-Dimensional Newton's Method

Overview

This assignment builds a multidimensional Newton solver and applies it to compute the hydrostatic structure of a white dwarf star. The star is modeled as n concentric mass shells with a polytropic equation of state; the unknowns are the shell-edge radii $\{r_i\}_{i=1}^n$.

Part 1: Multi-Dimensional Newton Solver — `newton_nd`

Problem Statement

Implement `newton_nd(F, J, x0, ...)` that solves the vector equation $F(\mathbf{x}) = \mathbf{0}$ using Newton's method. $F: \mathbb{R}^n \rightarrow \mathbb{R}^n$ is the residual vector; $J = \partial F / \partial \mathbf{x}$ is the Jacobian matrix. An optional `accept_step` callback can damp the Newton step to keep the solution in a physically valid region.

Algorithm

At each iteration, Newton's method linearizes F around the current guess $\mathbf{x}^{(k)}$:

$$J(\mathbf{x}^{(k)}) \Delta \mathbf{x} = -F(\mathbf{x}^{(k)})$$

Solve this linear system for $\Delta \mathbf{x}$, then update $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha \Delta \mathbf{x}$, where $\alpha \in (0, 1]$ is the damping factor from `accept_step`.

Convergence criteria: Stop when $\|F(\mathbf{x})\|_\infty < \varepsilon_F$ (residual small) or $\|\alpha \Delta \mathbf{x}\|_\infty < \varepsilon_x \|\mathbf{x}\|_\infty$ (step small relative to current value).

Quadratic convergence: Near the root, Newton's method satisfies $\|\mathbf{x}^{(k+1)} - \mathbf{x}^*\| \lesssim C \|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2$. In the test case (exact solution $(1, 2)$), the residual history $[0.96, 0.049, 4.7 \times 10^{-4}, 6.9 \times 10^{-8}, 1.8 \times 10^{-15}]$ shows the characteristic quadratic doubling of significant digits.

Implementation:

```
for iteration in range(max_iter):
    fx = np.asarray(F(x), dtype=float)
    if np.linalg.norm(fx, ord=np.inf) < tol_f:
        return x, {"converged": True, ...}
    jac = np.asarray(J(x), dtype=float)
    dx = np.linalg.solve(jac, -fx)      # solve J Δx = -F
    factor = 1.0 if accept_step is None else float(accept_step(x, dx))
    x = x + factor * dx
```

`np.linalg.solve` uses LU decomposition internally — $O(n^3)$ per iteration.

Part 2: The White Dwarf Model

Physical Setup

A white dwarf is supported against gravity by **electron degeneracy pressure**, modeled as a polytropic equation of state:

$$p = \kappa \rho^\gamma$$

with $\gamma = 5/3$ (non-relativistic) or $\gamma = 4/3$ (ultra-relativistic).

The star is divided into n mass shells. The mass grid is **fixed** and uniform:

$$m_i = i \cdot M_{\text{total}}/n, \quad i = 0, 1, \dots, n$$

The shell-edge radii $\{r_1, \dots, r_n\}$ are the **unknowns**. From the radii, density and pressure in each shell follow:

$$\rho_i = \frac{\Delta m_i}{(4\pi/3)(r_i^3 - r_{i-1}^3)}, \quad p_i = \kappa \rho_i^\gamma$$

The Residual Equations

Hydrostatic equilibrium (pressure gradient balances gravity) in finite-difference form gives one equation per shell interface:

$$f_i = \frac{p_{i+1} - p_i}{\Delta m_i^*} + \frac{Gm_i}{4\pi r_i^4} = 0, \quad i = 1, \dots, n$$

This is a **forward** difference of the hydrostatic equation $dp/dm = -Gm/(4\pi r^4)$. Here $\Delta m_i^* = \frac{1}{2}(m_{i+1} - m_{i-1}) = M/n$ is the mass associated with vertex i , and the pressure just outside the outermost shell is set to zero, $p_{n+1} \equiv 0$ (surface boundary condition: zero pressure at the star's edge).

The second term is the gravitational acceleration term. The code also uses a **scaled residual** (dividing f_i by the gravity term) so all components have magnitude ~ 1 near the solution, which improves Newton convergence.

The Analytic Jacobian

$\partial f_i / \partial r_j$ is derived analytically by differentiating the density and pressure formulas. Since $p_k = \kappa \rho_k^\gamma$, the chain rule gives:

$$\frac{\partial p_k}{\partial r_j} = \gamma \frac{p_k}{\rho_k} \cdot \frac{\partial \rho_k}{\partial r_j}, \quad \frac{\partial \rho_k}{\partial r_k} = -\frac{3\rho_k r_k^2}{r_k^3 - r_{k-1}^3}, \quad \frac{\partial \rho_k}{\partial r_{k-1}} = +\frac{3\rho_k r_{k-1}^2}{r_k^3 - r_{k-1}^3}$$

which yields the pressure derivatives actually assembled in the code:

$$\frac{\partial p_k}{\partial r_k} = -\frac{3\gamma p_k r_k^2}{r_k^3 - r_{k-1}^3}, \quad \frac{\partial p_k}{\partial r_{k-1}} = +\frac{3\gamma p_k r_{k-1}^2}{r_k^3 - r_{k-1}^3}$$

Each pressure p_k depends only on the two radii bounding its shell (r_{k-1} and r_k), and each residual f_i involves p_i and p_{i+1} ; therefore f_i depends on r_{i-1}, r_i, r_{i+1} and J is **tridiagonal**. The analytic Jacobian is verified against a finite-difference approximation; relative error $\approx 5 \times 10^{-11}$ confirms correctness.

Using the analytic Jacobian instead of a numerical one saves n extra residual evaluations per Newton step and avoids finite-difference truncation error.

Damping: Monotonic Radius Step

During early Newton iterations, an unconstrained step might make some radii decrease past each other (making shell volumes negative — unphysical). The `accept_step` callback `monotonic_radius_step_factor` finds the largest $\alpha \leq 1$ such that all shell widths remain positive after taking the step $r + \alpha \Delta r$. This keeps the solution in the feasible domain.

Results

Non-relativistic ($\gamma = 5/3$): The well-known analytical result is $R \propto M^{-1/3}$, or equivalently $M^{1/3}R = \text{const}$. The scan over $M = 0.4, 0.6, 0.8, 1.0, 1.2 M_\odot$ yields $M^{1/3}R \approx 0.01220$ for all masses, confirming the $M^{-1/3}$ scaling to 6 significant figures.

For $M = 1.0 M_\odot$: $R \approx 0.0122 R_\odot$, which is a realistic white dwarf radius.

Near-relativistic ($\gamma = 1.334$, approaching $4/3$): As $\gamma \rightarrow 4/3$, the polytropic model predicts a limiting Chandrasekhar mass $M_{\text{Ch}} \approx 1.44 M_\odot$ above which no equilibrium exists. The simulation shows the radius shrinking precipitously as $M \rightarrow M_{\text{Ch}}$: from $R \approx 557 R_\odot$ at $M = 1.41$ to $R \approx 0.05 R_\odot$ at $M = 1.45$, and non-convergence at $M = 1.46$. This sensitivity near the Chandrasekhar limit is a genuine physical effect captured by the model.

SciPy comparison: `scipy.optimize.root` with the `hybr` method (a variant of Powell’s hybrid method) gives the same radii to within the tolerances set, confirming the Newton implementation is correct.

Key Takeaways

- **Newton’s method generalizes to systems:** at each step solve the linear system $J \Delta \mathbf{x} = -F$ and update $\mathbf{x} \rightarrow \mathbf{x} + \Delta \mathbf{x}$. The Jacobian $J = \partial F / \partial \mathbf{x}$ replaces the scalar derivative.
- **Quadratic convergence** near the root — the number of correct digits roughly doubles each iteration — but only if the initial guess is good enough.
- **Prefer an analytic Jacobian** when you can derive it: it avoids finite-difference truncation error and saves n residual evaluations per step. Always verify it against a finite-difference check.
- **Exploit sparsity.** Here each residual couples only neighboring shells, so J is **tridiagonal** — cheap to solve. Recognizing structure is what makes large systems tractable.
- **Damping/globalization keeps Newton physical.** An undamped step can leave the valid domain (negative shell volumes); a step-limiting factor $\alpha \in (0, 1]$ keeps the iterate feasible.
- **Scale the residual** so all components are $O(1)$ near the solution — this conditions the linear system and improves convergence.
- **Physics insight:** the polytrope reproduces the white-dwarf mass-radius law $R \propto M^{-1/3}$ for $\gamma = 5/3$, and the Chandrasekhar instability (radius collapsing, solver failing to converge) as $\gamma \rightarrow 4/3$ — the numerics mirror the physics.

HW 05 — Adaptive Runge-Kutta ODE Solver and Baseball Trajectories

Overview

This assignment implements an adaptive Runge-Kutta ODE integrator with step-doubling error control and Richardson extrapolation, then applies it to simulate the trajectories of a baseball curveball and fastball including drag and Magnus force.

Part 1: Adaptive Runge-Kutta Solver

Problem Statement

Implement `rk_adaptive(f, t0, y0, t_end, tol, order)` that integrates the ODE $\mathbf{y}' = f(t, \mathbf{y})$ from t_0 to t_{end} with local error controlled to `tol`. Support both RK3 and RK4. Validate on $y' = -y$ (exact: e^{-t}) and a sum-of-Gaussians benchmark. Then validate on the baseball problem.

The Runge-Kutta Step

A single Runge-Kutta step of order p advances $\mathbf{y}$ from t to $t + h$ using a weighted combination of derivative evaluations (stages):

RK3 (Kutta's third-order method):

```
k1 = f(t, y)
k2 = f(t + h/2, y + (h/2)k1)
k3 = f(t + h, y - h·k1 + 2h·k2)
y_new = y + (h/6)(k1 + 4k2 + k3)
```

3 function evaluations. Local truncation error $O(h^4)$, global error $O(h^3)$.

RK4 (the classical 4th-order Runge-Kutta):

```
k1 = f(t, y)
k2 = f(t + h/2, y + (h/2)k1)
k3 = f(t + h/2, y + (h/2)k2)
k4 = f(t + h, y + h·k3)
y_new = y + (h/6)(k1 + 2k2 + 2k3 + k4)
```

4 function evaluations. Local truncation error $O(h^5)$, global error $O(h^4)$.

The weights come from the Butcher tableau, which ensures that the weighted sum matches the Taylor expansion of $\mathbf{y}(t + h)$ up to the specified order.

Step-Doubling Error Control

To estimate the local error without knowing the exact solution, the method takes: 1. One step of size h : gives $\mathbf{y}_{\text{full}}$. 2. Two steps of size $h/2$: gives $\mathbf{y}_{\text{half}}$.

The difference $\|\mathbf{y}_{\text{half}} - \mathbf{y}_{\text{full}}\|_{\infty}$ estimates the local truncation error (it scales as h^{p+1} for order p).

- If `err < tol`: accept the step, advance t , and try to increase h .
- If `err > tol`: reject the step, shrink h , and retry.

Step size adjustment formula (based on the $p + 1$ error scaling):

$$h_{\text{new}} = h \cdot \min \left(2, 0.9 \cdot \left(\frac{\text{tol}}{\text{err}} \right)^{1/(p+1)} \right)$$

The factor 0.9 is a safety margin; the factor 2 prevents the step from growing too fast.

Richardson Extrapolation

After accepting a step, rather than returning $\mathbf{y}_{\text{half}}$ (which has error $O(h^{p+1})$), the code applies Richardson extrapolation to get a higher-order estimate:

$$\mathbf{y}_{\text{best}} = \mathbf{y}_{\text{half}} + \frac{\mathbf{y}_{\text{half}} - \mathbf{y}_{\text{full}}}{2^p - 1}$$

This cancels the leading error term, giving an answer accurate to $O(h^{p+2})$ instead of $O(h^{p+1})$. For RK3 ($p = 3$), the denominator is $2^3 - 1 = 7$; for RK4 ($p = 4$), it is $2^4 - 1 = 15$.

Implementation:

```
scale = 2**order - 1
...
y_full, _ = rk_step(f, t, y, h, order)
y_mid, _ = rk_step(f, t, y, h/2, order)
y_half, _ = rk_step(f, t + h/2, y_mid, h/2, order)
err = np.max(np.abs(y_half - y_full))

if err <= tol:
    y = y_half + (y_half - y_full) / scale # Richardson extrapolation
    t += h
    h *= min(2.0, 0.9 * (tol / err)**(1.0 / (order + 1)))
else:
    h *= max(0.1, 0.9 * (tol / err)**(1.0 / (order + 1)))
```

Validation Results

Exponential decay $y' = -y$, $y(0) = 1$, **exact** $y(5) = e^{-5}$:

Order	tol	Error	Steps
RK3	10^{-3}	2.2×10^{-5}	12
RK3	10^{-9}	4.1×10^{-11}	249
RK4	10^{-3}	8.5×10^{-6}	10
RK4	10^{-9}	1.6×10^{-11}	86

RK4 reaches the same accuracy as RK3 with fewer steps, consistent with its higher order.

Sum-of-Gaussians benchmark: Five narrow Gaussians (width $a = 0.1$) centered at $x = -5, -1, 1, 4, 7$; exact integral is 5. The adaptive solver concentrates steps near each Gaussian and takes large steps in between, achieving high accuracy efficiently.

Part 2: Baseball Trajectory

Problem Statement

Simulate a baseball thrown from a pitcher to a batter (distance 60 ft = 18.29 m) subject to:
1. **Gravity** $-g\hat{z}$ 2. **Aerodynamic drag** (velocity-dependent) 3. **Magnus force** (from spin)

Compare a curveball ($v_0 = 85 \text{ mph} = 38.00 \text{ m/s}$, $\varphi = 45^\circ$) and a fastball ($v_0 = 95 \text{ mph} = 42.47 \text{ m/s}$, $\varphi = 225^\circ$) both with spin $\omega = 1800 \text{ rpm}$.

Equations of Motion

State vector: $\mathbf{y} = [x, y, z, v_x, v_y, v_z]$ (position + velocity, SI units).

The aerodynamic drag coefficient is velocity-dependent:

$$F(v) = 0.0039 + \frac{0.0058}{1 + e^{(v-35)/5}}$$

This is a smooth sigmoidal model: at low speeds the drag is dominated by the constant term (≈ 0.004); at high speeds the second term decreases the drag coefficient.

The equations of motion:

$$\dot{v}_x = -F(v) v v_x + B\omega(v_z \sin \varphi - v_y \cos \varphi)$$

$$\dot{v}_y = -F(v) v v_y + B\omega v_x \cos \varphi$$

$$\dot{v}_z = -g - F(v) v v_z - B\omega v_x \sin \varphi$$

where $B = 4.1 \times 10^{-4} \text{ m}^{-1}$ is the Magnus coefficient and φ is the spin axis orientation. The drag terms $-F(v) v v_i$ are opposite to each velocity component; the Magnus (cross-product) terms depend on the spin axis direction φ .

Initial conditions: Ball starts at origin with $v_x = v_0 \cos \theta$, $v_z = v_0 \sin \theta$ (small upward angle $\theta = 1^\circ$), $v_y = 0$.

Integration time: $t_f = L/v_0$ where $L = 18.288 \text{ m}$ (simplified — ignores slowing down, but the problem specifies this).

Implementation

```
def baseball_rhs(t, state, omega, phi):
    _, _, _, vx, vy, vz = state
    v = np.sqrt(vx**2 + vy**2 + vz**2)
    Fv = 0.0039 + 0.0058 / (1.0 + np.exp((v - 35.0) / 5.0))
    ax = -Fv*v*vx + B*omega*(vz*np.sin(phi) - vy*np.cos(phi))
    ay = -Fv*v*vy + B*omega*vx*np.cos(phi)
    az = -g - Fv*v*vz - B*omega*vx*np.sin(phi)
    return np.array([vx, vy, vz, ax, ay, az])
```

Results

Curveball ($\varphi = 45^\circ$): deflects $y \approx +0.22$ m sideways and drops $z \approx -1.01$ m. The large drop shows how gravity and downward Magnus combine.

Fastball ($\varphi = 225^\circ$, opposite spin axis): deflects $y \approx -0.20$ m (opposite direction) and drops only $z \approx -0.37$ m. The smaller drop is partly because the faster ball spends less time in flight, and the Magnus force has an upward component that partially counters gravity.

Accuracy vs. cost comparison: Running the curveball at reduced tolerances (10^{-1} to 10^{-7}) with both RK3 and RK4:

- RK4 at `tol=1e-1` gives $|\Delta y| \approx 1.4 \times 10^{-11}$ m from the high-accuracy reference — effectively machine precision — with only 84 function calls.
- RK3 needs `tol=1e-7` (135 calls) to reach similar accuracy.

This demonstrates that for smooth ODEs like this one, RK4's higher order per evaluation makes it significantly more efficient than RK3.

Key Takeaways

- **Runge-Kutta methods** build a high-order step from several derivative evaluations (stages) whose weights are chosen to match the Taylor expansion. RK4 (4 stages, global $O(h^4)$) is the standard workhorse.
- **Step-doubling estimates the local error for free:** compare one step of size h against two of size $h/2$; their difference scales as h^{p+1} and serves as the error estimate driving step-size control.
- **Adaptive step-size control:** accept if `err < tol`, else shrink and retry; rescale by $(\text{tol}/\text{err})^{1/(p+1)}$ with a safety factor. The solver automatically takes small steps in fast regions and large steps in smooth ones.
- **Richardson extrapolation** combines the two estimates, $y_{\text{best}} = y_{\text{half}} + \frac{y_{\text{half}} - y_{\text{full}}}{2^p - 1}$, cancelling the leading error term to gain a free order of accuracy.
- **Higher order is more efficient for smooth problems:** RK4 reaches a target accuracy in far fewer steps/calls than RK3, even though each step costs more.
- **Reduce any ODE to first order** by stacking into a state vector (here $[x, y, z, v_x, v_y, v_z]$); the same scalar RK machinery then integrates the whole system.

HW 06 — 1D Lagrangian Hydrodynamics: Sod Shock Tube

Overview

This assignment implements a staggered Lagrangian finite-difference scheme to solve the 1D Euler equations of compressible gas dynamics. The test problem is the **Sod shock tube**: a membrane separating two regions of different pressure is removed at $t = 0$, and three wave structures emerge — a rarefaction fan, a contact discontinuity, and a shock. The simulation is run on four grid resolutions ($N = 34, 100, 334, 1000$) and compared against the analytic exact solution.

Problem Statement

Evolve the ideal-gas Euler equations in 1D on the domain $[0, 16]$ from $t = 0$ to $t = 2.5$, with initial condition:

$$(\rho, u, p) = \begin{cases} (1.0, 0, 1.0) & x < 8 \\ (0.125, 0, 0.1) & x > 8 \end{cases}$$

Rigid walls at both ends. Equation of state: $p = (\gamma - 1)\rho e$ with $\gamma = 1.4$.

Compare numerically measured feature speeds (shock, contact, rarefaction head and tail) against the exact Riemann solution, and check energy conservation.

The Physical Waves (Exact Solution)

The Sod problem produces three waves traveling at speeds determined by the Rankine-Hugoniot and isentropic relations:

Feature	Speed (exact)	Description
Rarefaction head	-1.183	Left edge of the fan; moves left at $-c_L$
Rarefaction tail	-0.070	Right edge of the fan; barely moves
Contact discontinuity	$+0.927$	Jump in ρ only; no pressure or velocity jump
Shock	$+1.752$	Compression wave; jumps in ρ , p , u all

The intermediate pressure p_* is the root of $f_L(p_*) + f_R(p_*) + (u_R - u_L) = 0$, where each f_K is a branch (rarefaction or shock) depending on whether $p_* < p_K$ or $p_* > p_K$. This is solved with `brentq`.

The Numerical Scheme

Staggered Lagrangian Grid

The grid is **staggered**: two interleaved sets of degrees of freedom move with the fluid (Lagrangian frame):

- **Cells** ($i - \frac{1}{2}$): hold ρ, e, p, q (thermodynamic quantities, cell-centered).
- **Vertices** (i): hold x, u, a (position, velocity, acceleration).

Cell mass is conserved exactly: $m_{i-1/2} = \rho_{i-1/2} V_{i-1/2}$ is fixed. When vertices move, the cell volume changes and density is recomputed from $\rho = m/V$. This automatically satisfies mass conservation to machine precision — a key advantage of the Lagrangian approach.

Initial setup:

```
x = np.linspace(X_L, X_R, N + 1)  # N+1 vertices
xc = 0.5*(x[:-1] + x[1:])          # cell centres
rho = np.where(xc < X_DIAPH, RHO_L, RHO_R)
p = np.where(xc < X_DIAPH, P_L, P_R)
e = p / ((GAMMA - 1.0) * rho)
m = rho * np.diff(x)               # fixed cell masses
```

The Time-Step Algorithm (Leapfrog / Predictor-Corrector)

Each time step advances from t^n to t^{n+1} . The scheme is a **2nd-order predictor-corrector** (sometimes called staggered leapfrog):

Step 1 — Predictor velocity (half-step):

$$u_i^{n+1/2} = u_i^n + \frac{\Delta t}{2} a_i^n$$

Walls enforced: $u_0 = u_N = 0$.

Step 2 — New positions:

$$x_i^{n+1} = x_i^n + \Delta t u_i^{n+1/2}$$

Step 3 — New volumes and densities:

$$V_{i-1/2}^{n+1} = x_i^{n+1} - x_{i-1}^{n+1}, \quad \rho_{i-1/2}^{n+1} = m_{i-1/2} / V_{i-1/2}^{n+1}$$

Step 4 — Artificial viscosity (compression only — no viscosity in expanding regions):

$$q_{i-1/2}^{n+1} = \begin{cases} \sigma \rho^{n+1} (u_{i-1}^{n+1/2} - u_i^{n+1/2})^2 & u_{i-1} > u_i \text{ (compression)} \\ 0 & \text{otherwise} \end{cases}$$

The parameter $\sigma = 3$ controls the shock-smearing width. Artificial viscosity is necessary: without it, the shock would produce unphysical oscillations (Gibbs phenomenon) in the finite-difference scheme. It converts kinetic energy into thermal energy over a few cells near the shock, mimicking the real dissipation in a thin shock layer.

Step 5 — Energy update (algebraic implicit):

The internal energy evolves by work done against pressure and viscosity:

$$e^{n+1} = \frac{e^n - \frac{\Delta V}{2m} (p^n + q^n + q^{n+1})}{1 + (\gamma - 1) \frac{\Delta V}{2V^{n+1}}}$$

This form is **algebraically implicit** in e^{n+1} and $p^{n+1} = (\gamma - 1)\rho^{n+1}e^{n+1}$ simultaneously. For an ideal gas the system decouples analytically into a single formula, avoiding an iterative solve.

Step 6 — New accelerations:

$$a_i^{n+1} = -2 \frac{(p+q)_{i+1/2}^{n+1} - (p+q)_{i-1/2}^{n+1}}{m_{i-1/2} + m_{i+1/2}}$$

Interior vertices only; walls held at $a = 0$.

Step 7 — Corrector velocity:

$$u_i^{n+1} = u_i^{n+1/2} + \frac{\Delta t}{2} a_i^{n+1}$$

Time Step Control

The time step must satisfy the **Courant-Friedrichs-Lewy (CFL) condition** — no information can travel more than one cell per step:

$$\Delta t \leq \text{CFL} \cdot \min_i \frac{\Delta x_i}{c_i}$$

where $c_i = \sqrt{\gamma p_i / \rho_i}$ is the local sound speed. Additionally, a density-change limit prevents the density from changing by more than 5% per step:

$$\Delta t \leq \frac{0.05 \Delta x_i}{|\Delta u_i|}$$

A growth factor of 1.1 prevents the step from increasing too quickly after a refined region is passed.

```
dt = CFL * np.min(dx / c)
du = np.abs(s['u'][1:] - s['u'][:-1])
dt = min(dt, MAX_DRHO * np.min(dx[moving] / du[moving]))
dt = min(dt, DT_GROWTH * dt_prev)
```

Measuring Feature Speeds

Feature speeds are measured by locating each feature on two closely spaced snapshots (at $t = 2.0$ and $t = 2.2$) and computing $v = \Delta x / \Delta t$:

- **Shock:** rightmost crossing of the pressure level $\frac{1}{2}(p_* + p_R)$.
- **Contact:** crossing of the density level $\frac{1}{2}(\rho_{*L} + \rho_{*R})$ between the fan and the shock.
- **Rarefaction head:** crossing of $0.995 \rho_L$ (just inside the undisturbed left region).
- **Rarefaction tail:** crossing of $1.005 \rho_{*L}$ (just inside the star-left plateau).

Linear interpolation is used to locate the crossing precisely between grid points.

Results

Energy Conservation

N	$\Delta E/E_0$
34	6.4×10^{-5}
100	2.1×10^{-5}
334	6.3×10^{-6}
1000	2.1×10^{-6}

The energy non-conservation scales as N^{-1} (first-order), consistent with the first-order artificial viscosity scheme.

Feature Speeds vs. Exact

N	v_{shock}	v_{contact}	v_{head}	v_{tail}
34	1.795	1.046	-1.293	-0.495
100	1.717	0.941	-1.145	+0.024
334	1.763	0.928	-1.182	-0.076
1000	1.753	0.927	-1.176	-0.074
exact	1.752	0.927	-1.183	-0.070

The shock, contact, and head converge to the exact values at $N = 334$ -1000. The **tail** is the hardest feature to resolve: it moves at only -0.07 cm/s and sits inside the diffuse rarefaction fan. It requires $N \geq 334$ cells to even locate correctly.

Physical Interpretation

- The **rarefaction fan** is well-captured at all resolutions because it is a smooth expansion wave — finite differences handle smooth regions well.
- The **shock and contact** are smeared over $\sim \sigma = 3$ cell widths by the artificial viscosity; they sharpen as N increases.
- The shock reaches $x \approx X_{\text{diaph}} + v_{\text{shock}} \cdot t \approx 12.4$ at $t = 2.5$, well inside the domain $[0, 16]$, so the rigid walls never interfere with the solution.

Key Takeaways

- **Lagrangian grids move with the fluid**, so cell mass is conserved *exactly* (density is just $\rho = m/V$). This is the defining advantage over Eulerian (fixed-grid) schemes.
- **Staggered variables**: thermodynamic quantities (ρ, e, p) live at cell centers; kinematic quantities (x, u, a) live at vertices. This staggering gives natural second-order-accurate centered differences.
- **A predictor-corrector (leapfrog) time step** advances velocity at half-steps and positions/energy at full steps — second-order accurate in smooth regions.
- **Shocks need artificial viscosity**. Without it, finite-difference schemes produce spurious oscillations at discontinuities; the viscosity $q \propto \rho(\Delta u)^2$ (active only in compression) spreads each shock over a few cells and converts kinetic energy to heat.

- **The CFL condition** $\Delta t \leq \text{CFL} \cdot \min(\Delta x/c)$ is the stability limit for explicit schemes: information must not cross more than one cell per step.
- **Always validate against an exact solution.** The Sod problem has an analytic Riemann solution, so measured wave speeds and energy drift can be checked quantitatively.
- **Resolution trade-offs:** smooth features (rarefaction fan) converge fast; discontinuities (shock, contact) stay smeared by viscosity and sharpen only as N grows. Energy error falls $\sim N^{-1}$ — first-order, set by the viscosity at the shock.